Endpoint DLP for Windows — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Endpoint DLP for Windows in Microsoft Purview 4
- The problem . . . 4
- The solution . . . 4
- How it works — the model . . . 4
- What you get . . . 4
- Scope and boundaries . . . 4
- The bottom line . . . 5

Solution Overview — Endpoint DLP for Windows in Microsoft Purview 5
- What this solution delivers . . . 5
- What Endpoint DLP is (and is not) . . . 6
- The five egress channels this solution controls . . . 6
- Simulate first — this is a high-impact control . . . 7
- The standard deployment model . . . 7
- What's in the box . . . 7

Requirements — Endpoint DLP for Windows in Microsoft Purview 8
- R1. Endpoint DLP licensing, assigned to in-scope users . . . 8
- R2. Least-privilege roles for DLP + device onboarding . . . 8
- R3. Supported, healthy Windows devices ready to onboard . . . 9
- R4. A device-onboarding method chosen and available . . . 9
- R5. In-scope sensitive content to key the rules on (labels and/or SITs) . . . 9
- R6. Endpoint DLP global settings defined (device / printer / share groups, browsers, domains) . . . 9
- R7. The Microsoft Purview browser extension for Chrome/Firefox (cloud-egress + clipboard-to-web) 10
- R8. Per-channel control decision (audit / warn / block) — agreed with stakeholders . . . 10
- R9. A named pilot device group and change control (simulation/test first) . . . 10

Global Requirements — Microsoft Purview Solutions 11
- 1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 11
- 2. Purview roles (least-privilege) and portal access . . . 11
- 3. Security & Compliance PowerShell + Graph toolchain . . . 12
- 4. Data sources & scope in scope . . . 12
- 5. Compliance, records, privacy & legal stakeholders . . . 12
- 6. A pilot scope and change control . . . 12

Implementation Guide — Endpoint DLP for Windows in Microsoft Purview 13
- Phase 0 — Assess (confirm you are ready) . . . 14
- Phase 1 — Onboard the pilot Windows devices . . . 14
- Phase 2 — Configure the Endpoint DLP global settings . . . 14

Phase 3 — Author the policy and the five rules in SIMULATION . . . 15
Phase 4 — Pilot & validate in simulation . . . 16
Phase 5 — Enforce, then expand . . . 16
Rollback summary (fast reference) . . . 17
Testing & Validation — Endpoint DLP for Windows 17
A. Configuration gates (automated — `DLPWIN-Validate.ps1`, read-only) . . . 18
B. Behavioural tests — per channel (manual, on a pilot device) . . . 18
C. Negative / false-positive tests (both passes) . . . 19
D. Evidence to capture (for the defensibility pack) . . . 20
Troubleshooting — Endpoint DLP for Windows 20
1. A device shows no endpoint activity at all . . . 20
2. Some channels work, but Chrome/Firefox uploads are invisible . . . 21
3. The rule fires on the wrong things (false positives) . . . 21
4. The rule doesn't fire on content it should . . . 22
5. Simulation looks fine but enforcement doesn't block . . . 22
6. `New-DlpComplianceRule` fails on the endpoint restriction . . . 22
7. `-EndpointDlpLocation` / scope errors . . . 23
8. Activity report is empty or export fails . . . 23
9. "It blocked an admin / broke a workflow" — emergency rollback . . . 23
When to escalate to the determination owners (not a technical fix) . . . 24
Best Practices — Endpoint DLP for Windows 24
Deploy the way the house model says — simulate, then enforce . . . 24
Key the rules on *sensitive* content, not everything . . . 24
Get the endpoint global settings right — they are the vocabulary . . . 25
Onboarding hygiene . . . 25
Operate it, don't set-and-forget . . . 25
Keep the roles and determinations where they belong . . . 25
Security & handling of the tooling . . . 26
Deliverables Summary — Endpoint DLP for Windows in Microsoft Purview 26
Documents . . . 26
PowerShell tools (5) . . . 27
How it maps to the deployment model . . . 28
Definition of done . . . 28
Appendix A — PowerShell Scripts 29

Executive Summary — Endpoint DLP for Windows in Microsoft Purview

The problem

Sensitive data leaves Windows laptops through **local** channels that server-side controls never see: copied to a **USB stick**, sent to a **printer**, lifted through the **clipboard** into personal webmail, dropped onto an **unmanaged network share**, or **uploaded** to a personal cloud account in a browser. Email and SharePoint DLP are blind to all of it — these actions happen entirely on the device. Left unaddressed, that is both a **compliance gap** and a direct **data-risk exposure**.

The solution

This solution delivers **Microsoft Purview Endpoint DLP for Windows** end to end. It **onboards** Windows devices so those on-device activities become visible to Purview, configures the endpoint settings that define sanctioned USB devices / printers / shares / browsers / cloud domains, and authors a policy with rules for the **five egress channels — USB, print, clipboard, network share, and cloud egress** — keyed to your sensitive content (sensitivity labels and/or sensitive information types). Crucially, every rule is **piloted in simulation/test mode first** — it records what it *would* do without blocking anyone — validated on a small device ring, and only then moved to **enforce**.

How it works — the model

The delivery follows a disciplined, repeatable model with an explicit pass/fail gate at each step:

Assess → **Design** → **Configure** → **Pilot (simulation)** → **Validate** → **Expand (enforce).**

Because endpoint block actions stop work at the moment of the action, nothing is enforced until it has been proven, in simulation, to see the right channels and act only on the right content — with a one-command rollback available at all times.

What you get

- A complete document set: Overview, Requirements, an ordered Implementation guide, Testing (pass/fail gates), Troubleshooting, Best Practices, and this summary.
- **Five PowerShell tools** (Security & Compliance PowerShell / Microsoft Graph) — discovery, configuration (simulation, `-WhatIf` first), validation, reporting, and scope/health — all **read-only or preview by default, with no credentials in code.**
- A **defensibility evidence pack**: proof of what was configured, that each channel was validated in simulation before enforcement, and that enforcement behaves as designed.

Scope and boundaries

- Endpoint DLP is **licensing-dependent** (typically Microsoft 365 E5 / E5 Compliance / E5

Information Protection & Governance) and the Purview portal **evolves** — capability availability and exact paths are **verified against your licensing/tenant**, never assumed.

- **Regulatory, records, and legal determinations** — which data is sensitive, which channels must be blocked versus audited, and whether an action is a violation — remain with your **compliance, security, records, privacy, and legal** functions. This solution provides the **technical method and the evidence**, not the determination.

The bottom line

A set of **uncontrolled local exfiltration paths** on Windows endpoints becomes a **monitored, then enforced** control set in Microsoft Purview — **configured, piloted in simulation, validated, and defensible.**

Solution Overview — Endpoint DLP for Windows in Microsoft Purview

What this solution delivers

Sensitive data does not only leave the organization over email and SharePoint links. On a Windows laptop it walks out the **local** doors: a file copied to a **USB stick**, sent to a **printer**, lifted through the **clipboard** into a personal webmail tab, dropped onto an unmanaged **network share**, or **uploaded** to a personal cloud account in a browser. Server-side DLP (Exchange, SharePoint, OneDrive, Teams) never sees any of it — those actions happen entirely on the endpoint. **Microsoft Purview Endpoint DLP** closes that gap: once a Windows device is **onboarded**, Purview can **see and control** those on-device activities against your sensitive content.

This solution walks a competent-but-novice administrator through a **complete, repeatable Windows Endpoint DLP deployment**:

1. **Onboard Windows devices** to Microsoft Purview so the endpoint activities become visible — confirming the device prerequisites (supported Windows build, AV/onboarding health) and choosing an onboarding method (Intune, Group Policy, local script, or Configuration Manager).

1. **Configure the Endpoint DLP global settings** that every endpoint policy depends on — the pieces that define *what counts as* a USB device, a corporate printer, an allowed browser, or a sanctioned cloud domain (removable-media / printer / network-share groups, browser and service domain restrictions, restricted apps, file-path exclusions).

1. **Author a Windows Endpoint DLP policy** with rules for the five egress channels — **USB / removable media**, **print**, **clipboard**, **network share**, and **cloud egress** (browser upload) — each keyed to your sensitive content (sensitivity labels and/or sensitive information types), **piloted in simulation/test mode first** so nothing is blocked until you have validated it.

1. **Validate, then expand** — prove that each channel is seen and *would* act (audit → warn → block) on a small pilot device group, then move rules from simulation to **enforce** and widen scope deliberately, with an explicit pass/fail gate and a documented rollback at every step.

The result turns a set of **uncontrolled local exfiltration paths** into a **monitored, then enforced** control set on Windows endpoints — configured, piloted, validated, and defensible in Microsoft Purview.

What Endpoint DLP is (and is not)

• **It is** the extension of Purview Data Loss Prevention to the **Devices** location — the ability

to detect and act on **on-device** activities (copy to USB, print, clipboard, copy to network share, upload/egress via browser, and related activities) against content that matches your DLP conditions.

• **It requires onboarding.** A Windows device must be **onboarded to Microsoft Purview** (the same

onboarding surface shared with **Microsoft Defender for Endpoint**) before any endpoint activity is visible or controllable. **No onboarding, no endpoint DLP.**

• **It is not** a substitute for your Exchange/SharePoint/OneDrive/Teams DLP — it is the **Devices**

location alongside them. A single DLP policy can cover multiple locations; this solution focuses on the **Devices** location for Windows.

• **It is licensing-dependent.** Endpoint DLP is a premium capability (typically **Microsoft 365

E5**, the** E5 Compliance**, or the** E5 Information Protection & Governance **add-on) and in-scope users must be licensed. Some endpoint activities and settings are newer or in preview and the portal evolves —** verify capability availability and exact portal paths against your licensing/tenant** rather than assuming (see `global-requirements-purview.md` §1).

The five egress channels this solution controls

Each is an **on-device activity** Endpoint DLP can detect and, in a rule, **Audit**, **Warn with override**, or **Block**:

Channel	On-device activity controlled
USB / removable media	Copying a matching file to a USB / removable storage device (with allow-listed *device groups* for corporate-issued media).
Print	Printing a matching file (with allow-listed *printer groups* for sanctioned printers).
Clipboard	Copying matching content to the clipboard for paste into another app / browser.
Network share	Copying a matching file to a network share (with allow-listed *network-share groups*).
Cloud egress (browser upload)	Uploading a matching file to, or accessing it from, an **unallowed browser** or a **non-sanctioned service/cloud domain** (service-domain and sensitive-service-domain restrictions).

> ***Browser coverage. Microsoft Edge** is covered natively. **Chrome** and **Firefox** require the **Microsoft Purview browser extension** to see and control cloud-egress and clipboard-to-web activity. Confirm*

*the extension is deployed to in-scope browsers — **verify against your tenant.***

Simulate first — this is a high-impact control

Endpoint DLP **block** actions stop real work at the moment someone tries to copy, print, or upload. A mis-scoped rule can block a legitimate user mid-task. Therefore **every** policy in this solution is deployed in **simulation / test mode first** (the DLP policy modes *Test with notifications* / *Test without notifications*; in the portal, *"Run the policy in simulation mode."*) — the policy **evaluates and records what it *would* do without blocking anyone** — validated on a small pilot device group, and only then moved to **enforce**. **Verify the exact simulation/test terminology in your tenant** (the portal wording evolves).

> ***Determinations are not yours to make.*** **Which* data is sensitive, *which* channels must be blocked versus merely audited, and *whether* a given user's action is a policy violation are **compliance / records / security / HR-legal determinations** — this solution delivers the technical configuration and the evidence, not the determination (see `global-requirements-purview.md` §5).*

The standard deployment model

Every step follows the same repeatable model, with an explicit **pass/fail gate** between phases:

Assess → Design → Configure → Pilot (simulation/test) → Validate → Expand (enforce).

- **Assess** — confirm licensing, least-privilege roles, device-onboarding readiness, the in-scope sensitive content, the pilot device group, and compliance/security stakeholder sign-off.
- **Design** — decide which channels **audit** vs **warn** vs **block**, define the device / printer / network-share groups and the browser/service-domain lists, name every object, and set the pilot scope.
- **Configure** — onboard devices, set the Endpoint DLP global settings, then build the policy and its five rules in **simulation/test mode**.
- **Pilot** — run in simulation on the pilot devices; confirm each channel is seen and *would* act without blocking anyone.
- **Validate** — run the pass/fail gates in `05-testing.md`.
- **Expand** — move rules from simulation to enforce and widen device scope **only** after the gate passes; roll back per `03-implementation.md` if a gate fails.

What's in the box

- This overview, a **Requirements** guide, an ordered **Implementation** guide, **Testing** (pass/fail gates), **Troubleshooting**, **Best Practices**, and a **Deliverables Summary** + **Executive Summary**.
- **Five PowerShell tools** (Security & Compliance PowerShell / Microsoft Graph): a **Discovery** inventory of onboarding readiness and existing endpoint DLP config, a **Configure/Remediation** helper that builds the policy + rules in **simulation (`-WhatIf` first)**, a **Validation** checker, a **Reporting** rollup of endpoint DLP configuration and activity, and a **scope/health** helper — **read-only or `-WhatIf` by default; no credentials in code.**

> ***Sources.*** *Microsoft Learn — *Learn about Microsoft Purview Endpoint data loss prevention*, *Get started with / Onboard Windows devices into Microsoft Purview*, *Configure endpoint DLP settings*, *Create and deploy data loss prevention policies* (conditions, actions, simulation mode), and the *Microsoft Purview browser extension*. Endpoint DLP is **licensing-dependent** and the Purview portal is **evolving — verify capability availability, portal paths, and cmdlet parameters against your licensing/tenant** rather than assuming.*

Requirements — Endpoint DLP for Windows in Microsoft Purview

Confirm every prerequisite below **before** you configure anything. Category-level prerequisites (tenant, compliance licensing model, least-privilege roles, Security & Compliance PowerShell / Graph toolchain, in-scope data sources, compliance/privacy/legal stakeholders, pilot scope + change control) are in `global-requirements-purview.md` — reference them; they are **not** repeated here. The items below are the ones **specific to Windows Endpoint DLP**: **device onboarding**, the **endpoint global settings**, and the **per-channel** control decisions.

Each requirement is written as **Description · Why · Verify · Implement if missing.**

R1. Endpoint DLP licensing, assigned to in-scope users

Description: The licensing that includes **Endpoint DLP** — typically **Microsoft 365 E5**, the **E5 Compliance**, or the **E5 Information Protection & Governance** add-on — assigned to the users who sign in to the **in-scope Windows devices**. **Why:** Endpoint DLP is a premium, plan-gated capability; without the right licence the **Devices** location and the endpoint activities are not available, and endpoint enforcement is licensed **per user**. **Verify:** Microsoft 365 admin center → **Billing** → **Licenses** confirms the SKU/add-on. In the Purview portal, **Data Loss Prevention** exposes the **Devices** location and **Settings** → **Device onboarding** is present. Cross-check advanced endpoint activities against the *Microsoft Purview service description* — **verify against your licensing/tenant. Implement if missing:** Acquire and assign the required SKU/add-on to in-scope users before onboarding devices or authoring endpoint rules.

R2. Least-privilege roles for DLP + device onboarding

Description: The least-privilege roles, mapped to **named** people:

- **Compliance Administrator** or **Compliance Data Administrator** (or the **DLP Compliance Management** role group) to author the DLP policy and rules and to configure Endpoint DLP settings.
- The rights to **onboard devices** by your chosen method (e.g. **Intune Administrator** for an Intune configuration profile, or a device/GPO administrator for Group Policy / local script).
- **Global Administrator** only where genuinely required (e.g. first-time service enablement).

Why: Purview uses fine-grained role groups; policy authoring and device onboarding are distinct duties and should not be over-granted to one account. **Verify:** Purview portal → **Roles & scopes / Permissions** confirms the DLP role membership; your device-management console confirms the onboarding role. `DLPWIN-Discovery.ps1` reports the DLP policy/role signals it can read. **Implement if missing:** Assign

the least-privilege role for each duty under change control.

R3. Supported, healthy Windows devices ready to onboard

Description: In-scope endpoints running a **supported Windows build** (Windows 10 version 1809 or later, or Windows 11) with a current **antimalware client** and the servicing/updates Endpoint DLP requires, joined/managed in a way that supports your onboarding method. **Why:** Endpoint DLP sensing depends on the Windows client components; unsupported or unpatched devices will not report endpoint activities reliably. **Verify:** Confirm the OS build and update state on a sample of devices; `DLPWIN-Discovery.ps1` (Graph) reports managed-device OS/version signals where readable. **Confirm exact minimum build/AV requirements against current Microsoft docs for your tenant. Implement if missing:** Patch/upgrade devices to a supported build and healthy AV state before onboarding.

R4. A device-onboarding method chosen and available

Description: A chosen method to **onboard Windows devices to Microsoft Purview** — one of **Microsoft Intune** (endpoint DLP / device onboarding configuration profile), **Group Policy**, **local onboarding script**, or **Microsoft Configuration Manager** — using the onboarding package from **Purview** → **Settings** → **Device onboarding**. (Onboarding is shared with **Microsoft Defender for Endpoint**; devices already onboarded to Defender for Endpoint may already satisfy this.) **Why:** Onboarding is what makes endpoint activities visible to Purview; the method must fit how the estate is managed and be repeatable for the pilot and expansion rings. **Verify:** Purview → **Settings** → **Device onboarding** shows the onboarding packages/options; a test device appears in the **Devices** list after onboarding. `DLPWIN-ScopeHealth.ps1` prompts to confirm onboarding status in-portal and reports managed-device counts (Graph) as a proxy. **Implement if missing:** Select the onboarding method that matches your management tooling, download the package, and onboard the **pilot** devices first.

R5. In-scope sensitive content to key the rules on (labels and/or SITs)

Description: The **sensitive content** the endpoint rules will match — **sensitivity labels** (e.g. *Confidential*) already published and in use, and/or **sensitive information types** (built-in or custom) and/or trainable classifiers — that define *what* triggers the USB/print/clipboard/ network/cloud rules. **Why:** An endpoint rule acts on **content that matches a condition**; without labels or SITs to match, the rule has nothing to key on (or would apply to *all* content, which is rarely intended). **Verify:** Purview → **Information Protection** → **Labels** shows the labels published; **Data classification** → **Sensitive info types** shows the SITs; content in scope carries labels (Activity/ Content explorer where available). `DLPWIN-Discovery.ps1` inventories labels and SITs. **Implement if missing:** Publish/adopt the needed sensitivity labels or select the SITs first. (A full labeling rollout may be a separate project; endpoint rules can start on SITs and add label conditions once labels are in use.)

R6. Endpoint DLP global settings defined (device / printer / share groups, browsers, domains)

Description: The **Endpoint DLP settings** every endpoint rule depends on, defined with your stakeholders and named:

- **Removable-storage device groups** (which USB devices are corporate/allowed vs restricted).

- **Printer groups** (which printers are sanctioned).
- **Network-share coverage / groups** (which shares are in scope or excluded).
- **Browser and domain restrictions** — **unallowed browsers**, **service domains** (allowed) and **sensitive service domains**, and **unallowed cloud service domains** for cloud-egress control.
- **Restricted apps / app groups**, **file-path exclusions**, and (where used) **VPN settings** and **business-justification** options. **Why:** These settings are the *vocabulary* the rules speak — "block copy to USB **except** the corporate device group," "block upload **except** to sanctioned service domains." Without them the rules cannot distinguish sanctioned from unsanctioned paths. **Verify:** Purview → **Data Loss Prevention** → **Endpoint DLP settings** shows each group/list defined. `DLPWIN-Discovery.ps1` reports the endpoint settings it can read and flags anything not readable. **Implement if missing:** Define the groups and lists **before** authoring the rules that reference them. **Verify the exact settings pages and option names against your tenant** (they evolve).

R7. The Microsoft Purview browser extension for Chrome/Firefox (cloud-egress + clipboard-to-web)

Description: The **Microsoft Purview browser extension** deployed to **Chrome** and **Firefox** on in-scope devices, so cloud-egress and clipboard-to-web activity is visible in those browsers. (**Microsoft Edge** is covered natively and needs no extension.) **Why:** Without the extension, Chrome/Firefox uploads and web-paste actions are **not** seen or controlled — a coverage gap that undermines the cloud-egress rule. **Verify:** Confirm the extension is installed/managed on a sample of Chrome and Firefox instances (e.g. via your browser management policy). **Verify deployment method against current Microsoft docs.** **Implement if missing:** Deploy the extension (via Intune/GPO or the browser's enterprise policy) to in-scope Chrome/Firefox before relying on the cloud-egress rule for those browsers.

R8. Per-channel control decision (audit / warn / block) — agreed with stakeholders

Description: A documented decision, **per channel** (USB, print, clipboard, network share, cloud egress), of the intended action — **Audit**, **Warn with override**, or **Block** — and which sensitive content each applies to, agreed with **compliance / security** (and HR/legal where the action affects users). **Why:** The action is a **policy decision, not a technical default**; blocking versus auditing a channel has real operational and user impact and must reflect enterprise policy, not the administrator's preference. **Verify:** Confirm the per-channel action matrix and the in-scope content **in writing** before you move any rule from simulation to enforce. **Implement if missing:** Capture the decision with the relevant owners; until then, deploy **audit / simulation only.** (See `global-requirements-purview.md` §5.)

R9. A named pilot device group and change control (simulation/test first)

Description: A **named pilot device group** (a limited set of onboarded devices, e.g. `DG-EDLP-Pilot`, or a pilot user group `SG-EDLP-Pilot`) and agreement to run the policy in **simulation/test mode first**, under a **change-control** process, before any enforcing, broad-scope rollout. **Why:** Endpoint block actions are high-impact; simulation on a small ring proves the channels are seen and the rules *would* act **without** disrupting real work, and lets you tune before enforcing. **Verify:** Confirm the pilot group exists and is onboarded, that **simulation/test mode** is used, and a change window/approval path.

`DLPWIN-ScopeHealth.ps1` (`-WhatIf`) previews pilot-scope targeting. **Implement if missing:** Define the pilot group, onboard it, deploy in simulation, and agree change control before enabling enforcement.

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

• **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

• **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

• **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

• **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

• **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.

• **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.

• **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

• **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

• **Why required:** Configuration, automation, and the provided scripts run through these.

• **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.

• **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

• **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

• **Why required:** Purview policies key off the in-scope workloads and the classification scheme.

• **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

• **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

• **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

• **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

• **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.

• **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

• **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

• **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

• **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change window/approval path.

• **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

Implementation Guide — Endpoint DLP for Windows in Microsoft Purview

This is the ordered, do-it-in-this-sequence guide. It follows the standard model — **Assess** → **Design** → **Configure** → **Pilot (simulation/test)** → **Validate** → **Expand (enforce)** — with every object **named**, an explicit **pass/fail gate** at each phase, and a **rollback** for each change. Do **not** skip the simulation phase; endpoint block actions are high-impact.

Named objects used throughout (rename to your convention, but be consistent):

Object	Name used here
DLP policy	`EDLP-Windows-Pilot`
USB / removable-media rule	`EDLP-Rule-USB`
Print rule	`EDLP-Rule-Print`
Clipboard rule	`EDLP-Rule-Clipboard`
Network-share rule	`EDLP-Rule-NetworkShare`
Cloud-egress (browser) rule	`EDLP-Rule-CloudEgress`
Pilot device group	`DG-EDLP-Pilot`
Pilot user group	`SG-EDLP-Pilot`
Corporate removable-media device group	`USBG-Corporate`
Sanctioned printer group	`PRNG-Corporate`
In-scope network-share group	`NSG-Corporate`

> ***Tooling note.*** *Endpoint **device onboarding** and the **Endpoint DLP settings** (device/printer/ share groups, browsers, domains) are configured in the **Purview portal** — there is no supported Security & Compliance PowerShell cmdlet to *onboard* a device. The **policy and rules** can be authored in the portal **or** with* `New-/Set-DlpCompliancePolicy` *and* `New-/Set-DlpComplianceRule`*; the provided* `DLPWIN-Configure.ps1` *builds them in **simulation** with* `-WhatIf` *by default. **Verify every portal path and cmdlet parameter against your tenant** — the portal and the endpoint-restriction parameter names evolve.*

Phase 0 — Assess (confirm you are ready)

1. **Confirm requirements.** Walk `02-requirements.md` R1–R9. Do **not** proceed until licensing,

roles, supported devices, an onboarding method, in-scope sensitive content, the endpoint global settings plan, the browser extension plan, the per-channel action decision, and a pilot group are all confirmed.

1. **Run Discovery.** `Connect-IPPSSession`, then `.\DLPWIN-Discovery.ps1` to inventory existing DLP

policies/rules, the **Devices** coverage, sensitivity labels, and SITs, and to surface what is already in place. Keep the CSV as your baseline.

1. **Confirm onboarding readiness.** `.\DLPWIN-ScopeHealth.ps1` reports managed-device signals

(Graph) and prompts you to confirm, **in Purview** → **Settings** → **Device onboarding**, that the pilot devices are onboarded.

Gate 0 (pass/fail): All of R1–R9 satisfied, Discovery baseline captured, pilot devices onboarded and visible in **Purview** → **Devices**. **Fail** → **stop** and close the gap; do not configure.

Phase 1 — Onboard the pilot Windows devices

1. In **Purview portal** → **Settings** → **Device onboarding** (verify path), review the supported

onboarding methods and download the package for your chosen method (Intune / Group Policy / local script / Configuration Manager).

• *Local script* is fine for a **handful of pilot machines**; use **Intune or GPO** for scale.

• If devices are already **onboarded to Microsoft Defender for Endpoint**, they may already be

onboarded here — confirm they appear under **Devices**.

1. Onboard **only the pilot devices** (`DG-EDLP-Pilot`) first.

2. Deploy the **Microsoft Purview browser extension** to **Chrome/Firefox** on the pilot devices

(R7) if cloud-egress/clipboard-to-web coverage in those browsers is in scope. **Edge** needs no extension.

Gate 1: Pilot devices show as **onboarded** in **Purview** → **Devices**; a test activity (e.g. copy a benign file to a USB stick) appears in **Activity explorer** (Devices) within the expected latency. **Fail** → **rollback:** offboard the device via your onboarding tool's offboarding package; re-check the device prerequisites (R3) before retrying. **No policy is configured yet, so there is nothing to enforce-rollback at this stage.**

Phase 2 — Configure the Endpoint DLP global settings

In **Purview** → **Data Loss Prevention** → **Endpoint DLP settings** (verify path), define the vocabulary the rules will reference. Name each object per the table above.

1. **Removable storage device groups** — create `USBG-Corporate` listing the corporate-issued/allowed

USB devices (by device ID / vendor as your standard allows). This lets a USB rule **block everything except** approved media.

1. **Printer groups** — create `PRNG-Corporate` for sanctioned printers, so the print rule can allow corporate printing and control the rest.

1. **Network-share coverage / groups** — define `NSG-Corporate` (the shares in scope) and any exclusions.

1. **Browser and domain restrictions** — set **unallowed browsers** (browsers with no extension coverage that should be blocked from egress), **service domains** (sanctioned upload targets) and **sensitive service domains**, and **unallowed cloud service domains**. These drive the cloud-egress rule.

1. **Restricted apps / app groups**, **file-path exclusions**, and (if used) **VPN settings** and **business-justification** prompt text.

Gate 2: Each group/list above exists and is populated; `DLPWIN-Discovery.ps1` re-run reports them as readable (or you have confirmed them in-portal where a setting is not exposed to PowerShell). **Fail** → **rollback:** remove/blank the offending group; groups are inert until a rule references them, so there is no user impact yet.

Phase 3 — Author the policy and the five rules in SIMULATION

Create **one** DLP policy, `EDLP-Windows-Pilot`, scoped to the **Devices** location and the **pilot scope only**, in **simulation/test mode** (nothing is blocked). Then add the five rules. Author in the portal or run `.\DLPWIN-Configure.ps1 -WhatIf` first, then `.\DLPWIN-Configure.ps1` (still simulation) to create the objects.

1. **Create the policy** (simulation, Devices location, pilot scope):

• Location: **Devices** only, scoped to `DG-EDLP-Pilot` / `SG-EDLP-Pilot`.

• Mode: **simulation/test** (portal: *"Run the policy in simulation mode"*; PowerShell: a **Test** mode — `TestWithoutNotifications` for a silent pilot, or `TestWithNotifications` to also show policy tips). **Verify the exact mode value in your tenant.**

1. **Add the five rules**, each with a **condition** on your sensitive content (a sensitivity-label condition such as *Confidential*, and/or a sensitive-info-type condition) **and** the matching **endpoint restriction**:

Rule	Endpoint activity	Starting action	Allow-list group
`EDLP-Rule-USB`	Copy to a removable USB device	**Audit** (simulation)	except `USBG-Corporate`
`EDLP-Rule-Print`	Print	**Audit** (simulation)	except `PRNG-Corporate`
`EDLP-Rule-Clipboard`	Copy to clipboard (paste to app/browser)	**Audit** (simulation)	—
`EDLP-Rule-NetworkShare`	Copy to a network share	**Audit** (simulation)	scope `NSG-Corporate`

Rule	Endpoint activity	Starting action	Allow-list group
`EDLP-Rule-CloudEgress`	Upload to / access from unallowed browser or non-sanctioned service domain	**Audit** (simulation)	except sanctioned service domains

• Set actions **per the R8 decision matrix**, but keep everything **Audit** while the policy is in

simulation — the *enforce* action (Warn/Block) is applied in Phase 5, not now.

• The endpoint restrictions are set on the rule (in the portal under the **Devices** activities,

or in PowerShell via the rule's endpoint-restriction parameter). **Verify the exact restriction setting names and permitted values (**`Audit` **/** `Warn` **/** `Block` **/** `Ignore`**) against** `Get-Help New-DlpComplianceRule -Full` **in your tenant** before scripting them.

1. **Save.** The policy is now evaluating on the pilot devices **without blocking anyone.**

Gate 3: `EDLP-Windows-Pilot` exists, is in **simulation/test**, scoped to **Devices + pilot only**, with all five rules present and conditioned on sensitive content. `DLPWIN-Validate.ps1` reports **PASS** for policy-exists, mode-is-simulation, scope-is-pilot, and five-rules-present. **Fail** → **rollback:** `Set-DlpCompliancePolicy -Identity EDLP-Windows-Pilot -Mode Disable` (or delete the policy with `Remove-DlpCompliancePolicy`); simulation blocks nothing, so impact is limited to telemetry.

Phase 4 — Pilot & validate in simulation

1. Let the policy run in simulation on `DG-EDLP-Pilot` for the agreed observation window.
2. Perform the **sanctioned test actions** in `05-testing.md` on a pilot device (copy a test-labeled

file to USB, print it, copy to clipboard into a browser, copy to an out-of-scope share, upload to a personal cloud domain in each in-scope browser).

1. Review **Activity explorer** (Devices) and the policy's **simulation results**; run

`.\DLPWIN-Report.ps1` to roll up the configuration and (where available) the endpoint activity via **Activity explorer export**.

1. **Tune:** adjust conditions/allow-lists to remove false positives (e.g. add a legitimate corporate

USB to `USBG-Corporate`) and confirm every intended channel is **seen**.

Gate 4: Every one of the five channels registered the sanctioned test action in simulation (policy *would* act), false positives triaged, and the **R8 action matrix** confirmed **in writing**. **Fail** → **do not enforce**; keep tuning in simulation. Rollback is unchanged from Phase 3 (disable/ delete the policy).

Phase 5 — Enforce, then expand

Only after Gate 4 passes.

1. **Apply the enforce actions.** Update each rule to its **R8** action — **Warn (override)** or

Block — for the channels that require it; leave **Audit** where the decision is audit-only.

1. **Move the policy out of simulation** to **enforce** (portal: turn the policy **On**; PowerShell: `Set-DlpCompliancePolicy -Identity EDLP-Windows-Pilot -Mode Enable`). Do this **still scoped to the pilot** first.
1. **Re-run the sanctioned tests** (`05-testing.md`) and confirm the blocked/warned channels now **actually** block/warn on the pilot, with the expected policy tip and (if configured) business justification.
1. **Expand scope in rings.** Widen the **Devices** scope (add the next device group), re-validate, and repeat. Consider renaming/cloning to a production policy (`EDLP-Windows`) as you exit pilot, per your naming standard.

Gate 5: On the pilot, each **block/warn** channel enforces as designed and each **audit** channel records without blocking; expansion rings each pass the same gate before widening. **Fail** → **rollback:** immediately return the policy to **simulation/test** (`Set-DlpCompliancePolicy -Mode TestWithoutNotifications`) or **Disable** it — this stops enforcement at once — then diagnose per `06-troubleshooting.md` before re-enforcing.

Rollback summary (fast reference)

Situation	Action
A rule blocks legitimate work	Set that rule's action back to **Audit**, or set the policy to **simulation/test**, then tune the allow-list/conditions.
The whole policy misbehaves	`Set-DlpCompliancePolicy -Identity EDLP-Windows-Pilot -Mode Disable` (stops all enforcement) or `-Mode TestWithoutNotifications` (back to simulation).
A device should not be in scope	Remove it from the scoped device/user group, or **offboard** it via your onboarding tool's offboarding package.
Start over cleanly	`Remove-DlpComplianceRule` for each rule, then `Remove-DlpCompliancePolicy -Identity EDLP-Windows-Pilot`.

> *Enforcement changes are **high-impact** — make them under change control, keep the pilot ring small, and never move all channels to **Block** at once without validating each in simulation first.*

Testing & Validation — Endpoint DLP for Windows

This is the pass/fail gate set. Run it in two passes: **once in simulation** (Phase 4 — the channel is *seen* and the policy *would* act, nothing is blocked) and **again after enforce** (Phase 5 — the block/warn channels *actually* act). Every test uses **benign test content** deliberately marked to match your policy conditions (e.g. a document carrying your *Confidential* sensitivity label and/or seeded with a matching sensitive information type) — **never** real regulated data.

> ***Determination boundary.*** *These tests prove the **control works as configured**. Whether a given real-world action is a *violation*, and any consequence, is a **compliance / security / HR-legal** determination — not a test outcome (see* `global-requirements-purview.md` *§5).*

Run `.\DLPWIN-Validate.ps1` for the configuration gates and `.\DLPWIN-Report.ps1` for the activity/telemetry rollup between passes.

A. Configuration gates (automated — `DLPWIN-Validate.ps1`, read-only)

#	Check	PASS signal
A1	Policy exists	`EDLP-Windows-Pilot` is present.
A2	Devices location	The policy covers the **Devices** location.
A3	Simulation first	In the pilot pass, the policy **Mode** is a **Test/simulation** mode (not `Enable`).
A4	Pilot scope	The policy is scoped to `DG-EDLP-Pilot` / `SG-EDLP-Pilot`, **not** all devices.
A5	Five rules present	`EDLP-Rule-USB`, `-Print`, `-Clipboard`, `-NetworkShare`, `-CloudEgress` all exist and are enabled.
A6	Sensitive-content condition	Each rule carries a sensitivity-label and/or sensitive-info-type condition (not "all content").
A7	Allow-list groups referenced	USB/print/network rules reference `USBG-Corporate` / `PRNG-Corporate` / `NSG-Corporate` as designed.

Gate A: All applicable rows PASS. Any FAIL → fix in `03-implementation.md` before behavioural testing.

B. Behavioural tests — per channel (manual, on a pilot device)

For each channel, perform the action with a **matching** test file and record the result. In the **simulation pass** the expected result is *"activity recorded in Activity explorer / simulation results; no block."* In the **enforce pass** the expected result is the **R8** action for that channel.

#	Channel	Test action (on a pilot device, with a matching test file)	Simulation PASS	Enforce PASS (per R8)
B1	**USB**	Copy the test file to a **non-corporate** USB stick	Activity recorded; no block	Blocked/Warned as designed; copy to a `USBG-Corporate` device is **allowed**
B2	**Print**	Print the test file to a **non-sanctioned** printer	Activity recorded; no block	Blocked/Warned; printing to `PRNG-Corporate` is **allowed**
B3	**Clipboard**	Copy content from the test file and paste into a browser/other app	Activity recorded; no block	Blocked/Warned as designed
B4	**Network share**	Copy the test file to an **out-of-scope / unmanaged** network share	Activity recorded; no block	Blocked/Warned; copy to an in-scope share is handled as designed
B5	**Cloud egress — Edge**	Upload the test file to a **personal/non-sanctioned** cloud domain in **Edge**	Activity recorded; no block	Blocked/Warned; upload to a sanctioned service domain is **allowed**
B6	**Cloud egress — Chrome/Firefox**	Repeat B5 in **Chrome** and **Firefox** (extension deployed)	Activity recorded; no block	Blocked/Warned as in Edge — confirms extension coverage

Gate B (simulation pass): Every in-scope channel **B1–B6** recorded the activity. A channel that records **nothing** is a coverage failure — do not enforce it; diagnose per `06-troubleshooting.md` (onboarding, extension, latency, condition match).

Gate B (enforce pass): Every **block/warn** channel enforces as designed **and** every **allow-listed** exception (corporate USB, sanctioned printer, sanctioned domain) is **not** blocked. Any deviation → roll the policy back to simulation and tune.

C. Negative / false-positive tests (both passes)

#	Check	PASS signal
C1	Non-matching content	Copying/printing/uploading a file that carries **no** matching label/SIT triggers **no** endpoint match.

#	Check	PASS signal
C2	Allow-listed paths	The corporate USB (`USBG-Corporate`), sanctioned printer (`PRNG-Corporate`), and sanctioned service domains are **not** blocked in the enforce pass.
C3	Out-of-scope device	A device **not** in `DG-EDLP-Pilot` shows **no** endpoint enforcement from this policy.
C4	Override/justification	Where **Warn/override** or **business justification** is configured, the prompt appears and the override is **logged**.

Gate C: All applicable rows PASS. A false positive (C1/C2 fails) must be tuned **before** wider enforcement — false positives erode user trust and drive shadow workarounds.

D. Evidence to capture (for the defensibility pack)

Capture and retain, for each pass:

- `DLPWIN-Validate.ps1` output (configuration gate results).
- `DLPWIN-Report.ps1` output (policy modes + Activity-explorer export of endpoint activities, where available).
- Screenshots of **Purview** → **Activity explorer (Devices)** showing each channel's recorded activity, and of the **policy tip / block dialog** in the enforce pass.
- A completed **B-channel matrix** signed off by the change owner, plus the **R8 action decision** and the **onboarding confirmation** (pilot devices listed in **Purview** → **Devices**).

> *This evidence set is what makes the control **defensible** — it shows *what* was configured, *that* each channel was validated in simulation before enforcement, and *that* enforcement behaves as designed. Store it with the change record.*

Troubleshooting — Endpoint DLP for Windows

The failures you will actually hit, with the most likely cause and the fix. Work top-down: **most "the policy isn't working" reports trace back to onboarding, browser coverage, condition matching, or simulation-vs-enforce confusion — not the rule itself.** Where a portal path or cmdlet parameter is named, **verify it against your tenant** — the Purview portal and endpoint restriction options evolve.

1. A device shows no endpoint activity at all

Symptoms: Copy/print/upload a matching file on a device and **nothing** appears in Activity explorer (Devices); the policy never fires.

Likely causes & fixes

- **Device isn't onboarded.** The #1 cause. Confirm the device appears under **Purview** → **Devices** (Settings → Device onboarding). If not, re-run your onboarding method (Intune/GPO/local script/ConfigMgr). There is **no** S&C PowerShell cmdlet to onboard — it is a portal/management-tool action.
- **Unsupported / unpatched Windows build.** Endpoint sensing needs a supported build (Windows 10 1809+ / Windows 11) and a current antimalware client (R3). Patch and retest.
- **Onboarding still propagating.** After onboarding, allow time for the device to register and for activity to surface — endpoint telemetry is **not** instant. Wait and retry.
- **Check the proxy signal:** `DLPWIN-ScopeHealth.ps1` reports managed-device counts (Graph) — but remember that is a **proxy**; the authoritative check is the **Devices** list in the portal.

2. Some channels work, but Chrome/Firefox uploads are invisible

Symptoms: Cloud-egress is caught in **Edge** but not in **Chrome** or **Firefox**.

Fix

- **Deploy the Microsoft Purview browser extension** to Chrome/Firefox (R7). Edge is native; Chrome/Firefox **cannot** see cloud-egress or clipboard-to-web without the extension.
- Until the extension is deployed, treat those browsers as **unallowed browsers** in the Endpoint DLP settings so egress through them is controlled rather than silently uncovered.
- Verify the extension is actually installed/managed on a sample device (via your browser enterprise policy).

3. The rule fires on the wrong things (false positives)

Symptoms: Legitimate work is flagged/blocked — a corporate USB, a sanctioned printer, an approved SaaS upload.

Likely causes & fixes

- **Allow-list groups not referenced (or stale).** Add the corporate device to `USBG-Corporate`, the printer to `PRNG-Corporate`, or the domain to the sanctioned **service domains** list, and make sure the rule actually references the group.
- **Condition too broad.** If a rule keys on "all content" instead of a **sensitivity label / SIT**, it will match everything. Tighten the condition (R5). `DLPWIN-Validate.ps1` gate **A6** flags rules with no readable content condition.

- **SIT confidence/count too loose.** Raise the confidence level or instance count on the sensitive information type to cut noise, then re-test.
- Tune **in simulation** — never tune by loosening an already-enforcing block in production.

4. The rule doesn't fire on content it should

Symptoms: A file you expect to match is copied/printed with no policy match.

Likely causes & fixes

- **Content doesn't carry the label/SIT.** The test file must actually carry the **sensitivity label** or contain the **sensitive information type** the rule keys on. Confirm the label is applied / the SIT pattern is present (seed a known match).
- **Label not published / SIT not in scope.** Confirm the label is published to the user (Information Protection → Label policies) and the SIT exists (Data classification). `DLPWIN-Discovery.ps1` inventories both.
- **Rule disabled.** `DLPWIN-Validate.ps1` gate **A5b** flags disabled rules.

5. Simulation looks fine but enforcement doesn't block

Symptoms: In simulation the activity is recorded; after "turning it on" the block/warn still doesn't happen.

Likely causes & fixes

- **Policy still in a Test mode.** Simulation records but does **not** block. To enforce, move the policy out of simulation: portal → turn the policy **On**; PowerShell → `Set-DlpCompliancePolicy -Identity EDLP-Windows-Pilot -Mode Enable`. `DLPWIN-Validate.ps1` gate **A3** shows the current mode.
- **Rule action still Audit.** The rule's endpoint restriction may be **Audit** even though the policy is enforcing. Set the per-channel action to **Warn/Block** per the R8 decision.
- **Device out of scope.** Enforcement only applies to devices in the policy's Devices scope. Confirm with `DLPWIN-ScopeHealth.ps1` / gate **A4**.
- **Propagation delay.** Policy/action changes take time to reach endpoints. Wait, then retest.

6. `New-DlpComplianceRule` fails on the endpoint restriction

Symptoms: `DLPWIN-Configure.ps1 -Commit` (or manual scripting) errors when setting the endpoint restriction, e.g. an invalid Setting name or value.

Fix

- **Verify the exact Setting names and values for your tenant.** The endpoint-restriction Setting

names (e.g. removable media, print, clipboard, network share, cloud egress) and permitted values (`Audit` / `Warn` / `Block` / `Ignore`) **evolve**. Run `Get-Help New-DlpComplianceRule -Full` and check current Microsoft docs, then adjust the hashtable. The script deliberately **prints the planned Setting names** so you can verify before `-Commit`.

- **Prefer the portal for the first build** if PowerShell parameter names are uncertain, then read the result back with `DLPWIN-Discovery.ps1` to learn the exact shape in your tenant.

7. `-EndpointDlpLocation` / scope errors

Symptoms: Creating the policy errors on the location/scope value.

Fix

- The **Devices** location is scoped by user/group (or `All`). Confirm the pilot group (`SG-EDLP-Pilot`) exists and the identifier is one your tenant accepts. `DLPWIN-ScopeHealth.ps1` reports the group and the current scope.
- Avoid `All` during the pilot — scope to the pilot group first (gate **A4**).

8. Activity report is empty or export fails

Symptoms: `DLPWIN-Report.ps1` returns no activity, or the Activity Explorer export warns and skips.

Likely causes & fixes

- **Licensing/permission.** `Export-ActivityExplorerData` is licensing-dependent; the script **warns and degrades to policy-modes only **rather than failing. Fall back to** Purview → Activity explorer **(filter to** Devices**) in the portal.
- **Quiet pilot window.** A small pilot may simply have no activity in the window — widen `-Days` or perform the sanctioned test actions from `05-testing.md` first.
- **Role.** Ensure the connected account holds a role that can read Activity Explorer.

9. "It blocked an admin / broke a workflow" — emergency rollback

Do this immediately, then diagnose:

- Return the policy to **simulation** or **off**:

`Set-DlpCompliancePolicy -Identity EDLP-Windows-Pilot -Mode TestWithoutNotifications` (simulation) or `-Mode Disable` (off). Either **stops enforcement at once.**

- Or set the offending **rule's action back to Audit**.
- Then work the relevant section above (usually §3 false positives or §5 scope), fix **in simulation**, re-validate with `DLPWIN-Validate.ps1`, and only then re-enforce.

When to escalate to the determination owners (not a technical fix)

Some questions are **not** troubleshooting — they are **determinations** that belong to compliance / security / records / HR-legal (see `global-requirements-purview.md` §5):

- *Should* this channel be **blocked** or only **audited**? → the **R8** decision owners.
- Is *this content* actually sensitive / in scope? → **compliance / data owners**.
- Is a user's action a **violation**, and what is the consequence? → **HR / legal**.

Provide them the **evidence** (`DLPWIN-Report.ps1`, Activity explorer, the validation output) — not a verdict.

Best Practices — Endpoint DLP for Windows

Field-tested guidance for running Windows Endpoint DLP well. It complements — does not replace — the implementation steps and the pass/fail gates.

Deploy the way the house model says — simulate, then enforce

- **Never enforce a channel you have not validated in simulation.** The single most common cause of a

DLP incident is a **Block** rule that was moved to enforce without a simulation pass. Every channel goes **Audit/simulation** → **validate** → **enforce**, one ring at a time.

- **Small pilot ring first.** Onboard and scope a handful of devices (`DG-EDLP-Pilot`) before the

estate. It is far cheaper to find a false positive on ten machines than ten thousand.

- **One change at a time.** Do not flip all five channels to **Block** together. Enforce the

highest-risk channel first (often USB or cloud egress), validate, then the next.

- **Keep a rollback ready.** `Set-DlpCompliancePolicy -Mode TestWithoutNotifications` (back to

simulation) or `-Mode Disable` stops enforcement instantly — know the command before you enforce.

Key the rules on *sensitive* content, not everything

- **Condition on labels and/or SITs**, not "all content." An endpoint policy that blocks *any* USB

copy will generate noise and workarounds; one that blocks **labeled/classified** content is defensible and proportionate.

- **Lead with sensitivity labels where they exist** — a label is an explicit business decision about

sensitivity and travels with the file. Fall back to **sensitive information types** where labels are not yet adopted, and add label conditions as labeling matures.

- **Tune SIT confidence and count** to reduce false positives before you enforce.

Get the endpoint global settings right — they are the vocabulary

• **Maintain the allow-list groups deliberately.** `USBG-Corporate`, `PRNG-Corporate`, and `NSG-Corporate` are what let you **block the bad path without blocking the business**. Review them on a schedule; a stale allow-list is both a false-positive source and a bypass.

• **Cover all three browsers.** Edge is native; **Chrome and Firefox need the Microsoft Purview browser extension **or their cloud-egress/clipboard-to-web activity is invisible. An un-extended browser is an open door — treat it as an** unallowed browser** until the extension is deployed.

• **Curate service domains vs sensitive service domains** so sanctioned SaaS uploads are allowed and personal/consumer cloud is controlled. Keep the list current as the business adopts new tools.

• **Use file-path exclusions sparingly** — each exclusion is a monitoring blind spot.

Onboarding hygiene

• **Onboarding is shared with Defender for Endpoint** — coordinate so you do not double-manage or fight another team's onboarding/offboarding. Confirm devices in **Purview** → **Devices**, not by assumption.

• **Keep the Windows client and AV current** — endpoint sensing depends on it; unpatched devices under-report.

• **Offboard cleanly** using the offboarding package when a device leaves scope; do not just delete the record.

Operate it, don't set-and-forget

• **Review Activity explorer (Devices) regularly** and roll up with `DLPWIN-Report.ps1`. Rising activity on a channel is a signal to review scope, tuning, or user education — not automatically a reason to block harder.

• **Watch override / business-justification trends.** A channel that is constantly overridden may be mis-scoped or may indicate a genuine business need that belongs in an allow-list, not a block.

• **Version and name policies clearly.** Move from `EDLP-Windows-Pilot` to a production `EDLP-Windows` as you exit pilot; keep a change log of scope and action changes.

• **Re-validate after portal changes.** The Purview portal and the endpoint restriction options evolve; re-run `DLPWIN-Validate.ps1` after Microsoft feature updates and confirm nothing regressed.

Keep the roles and determinations where they belong

• **Least privilege, split duties.** DLP authoring and device onboarding are different roles — do not fold them into one over-privileged account.

• **Determinations stay with the owners.** *Which* data is sensitive, *which* channels to block, and *whether* an action is a violation are **compliance / security / HR-legal** calls. You provide the configuration and the evidence, not the judgement (see `global-requirements-purview.md` §5).

• **Communicate before you enforce.** Policy tips and user notice reduce friction and support the lawful, transparent operation of an endpoint control; surprise blocks generate tickets and workarounds.

Security & handling of the tooling

• **Read-only / `-WhatIf` by default.** Run the provided scripts in their safe default first; only add `-Commit`/`-Apply` when you intend to write, under change control.

• **No secrets in code.** The scripts use interactive modern-auth (`Connect-IPPSSession` / `Connect-MgGraph`); never hardcode credentials, tokens, or app secrets.

• **Least-privilege connections.** Connect with the least-privilege role that the task needs, not a standing Global Administrator session.

Deliverables Summary — Endpoint DLP for Windows in Microsoft Purview

Everything included in this solution, what each piece is for, and how it maps to the deployment model. Use it as the table of contents and the "did we finish?" checklist.

Documents

Document	What it gives you
01 — Solution Overview	What Endpoint DLP for Windows delivers, the five egress channels (USB, print, clipboard, network share, cloud egress), onboarding, the simulate-first rule, and the Assess→Design→Configure→Pilot→Validate→Expand model.
02 — Requirements	The solution-specific prerequisites (R1–R9): endpoint licensing, DLP + onboarding roles, supported devices, an onboarding method, in-scope sensitive content, the endpoint global settings, the browser extension, the per-channel action decision, and a pilot group — each with Verify / Implement-if-missing. References the category `global-requirements-purview.md`.

Document	What it gives you
03 — Implementation Guide	The ordered build: onboard pilot devices → set endpoint global settings → author the policy + five rules **in simulation** → pilot & validate → enforce & expand. Every object named, with a pass/fail gate and rollback per phase.
05 — Testing & Validation	The pass/fail gates: configuration gates (A), per-channel behavioural tests (B) run in simulation then enforce, false-positive/negative tests (C), and the evidence to capture (D).
06 — Troubleshooting	The failures you will actually hit — no endpoint activity, a channel invisible, Chrome/Firefox not covered, false positives, enforcement not applying, simulation confusion — each with cause and fix.
07 — Best Practices	How to run it well: simulate-then-enforce discipline, keying on sensitive content, maintaining the allow-list groups and browser coverage, onboarding hygiene, ongoing operation, and role/determination boundaries.
Executive Summary	The one-page business-level summary for stakeholders and sign-off.

PowerShell tools (5)

All are **read-only or** `-WhatIf` **by default**, use interactive modern-auth (`Connect-IPPSSession` / `Connect-MgGraph`), and contain **no credentials**. They do **not** onboard devices (onboarding is a portal/management-tool action) and do **not** invent cmdlets.

Script	Purpose	Safe default
DLPWIN-Discovery.ps1	Inventory existing DLP policies/rules and **Devices** coverage, sensitivity labels, and sensitive info types — the baseline of what already protects the endpoints.	Read-only
DLPWIN-Configure.ps1	Build/update the `EDLP-Windows-Pilot` policy and the five channel rules **in simulation**; requires `-Commit` to write and supports `-WhatIf`.	`-WhatIf` (preview)
DLPWIN-Validate.ps1	Run the configuration gates (policy exists, Devices location, simulation-first, pilot scope, five rules, sensitive-content conditions) with PASS/FAIL output.	Read-only

Script	Purpose	Safe default
DLPWIN-Report.ps1	Roll up policy modes and, where available, **endpoint DLP activity** via Activity-explorer export — the operational + evidence view.	Read-only
DLPWIN-ScopeHealth.ps1	Report onboarding/scope health (managed-device signals via Graph, policy scope) and preview pilot-scope targeting; `-WhatIf` for any scoping change.	`-WhatIf` / read-only

How it maps to the deployment model

Phase	Deliverables used
Assess	01, 02, `DLPWIN-Discovery.ps1`, `DLPWIN-ScopeHealth.ps1`
Design	02, 03 (named objects + action matrix), 07
Configure	03, `DLPWIN-Configure.ps1 -WhatIf` then `-Commit` (simulation)
Pilot (simulation)	03, 05 (Gate B simulation pass), `DLPWIN-Report.ps1`
Validate	05, `DLPWIN-Validate.ps1`, `DLPWIN-Report.ps1`
Expand (enforce)	03 (Phase 5), 05 (Gate B enforce pass), 06, 07

Definition of done

• Pilot Windows devices **onboarded** and visible in **Purview** → **Devices**.

• Endpoint global settings (device / printer / network-share groups, browsers, service domains) defined.

• `EDLP-Windows-Pilot` with five rules built **in simulation**, all configuration gates (A) PASS.

• Behavioural gates (B) PASS in the **simulation pass**, then in the **enforce pass** for the channels the R8 decision blocks/warns; false-positive gates (C) PASS.

• **Evidence pack** (D) captured and stored with the change record.

• Regulatory / policy determinations (which data, which action) recorded by the **compliance / security** owners — not by the administrator.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

DLPWIN-Configure.ps1

Purpose: Creates (or updates) a Windows Endpoint DLP compliance policy scoped to the Devices location and a pilot scope, in SIMULATION/TEST mode, plus five rules — USB/removable media, print, clipboard, network share, and cloud egress — each conditioned on sensitive content and set to Audit while in simulation. SAFE BY DEFAULT: previews with -WhatIf and writes NOTHING unless you pass -Commit. It never enforces (never sets Block/Enable) — moving to enforce is a deliberate portal/PowerShell step in Phase 5 of the implementation guide. HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\DLPWIN-Configure.ps1 -WhatIf # preview only (default is preview) .\DLPWIN-Configure.ps1 -Commit # create the policy + rules in SIMULATION .\DLPWIN-Configure.ps1 -Commit -PilotGroup "SG-EDLP-Pilot" -LabelName "Confidential" EXPECTED RESULTS With -Commit: policy 'EDLP-Windows-Pilot' exists in a Test/simulation mode, Devices location, pilot scope, with five Audit rules. Validate with DLPWIN-Validate.ps1 (expect Gate A PASS). Without -Commit: a printed plan and no changes. .NOTES White-label Microsoft Purview tooling. -WhatIf/preview by default; -Commit gates all writes. No credentials stored. Uses standard New-/Set-DlpCompliancePolicy and New-/Set-DlpComplianceRule; NO invented cmdlets. The exact endpoint-restriction Setting names/values and the simulation Mode value EVOLVE — the script uses documented names and PRINTS them for you to verify with `Get-Help New-DlpComplianceRule -Full` against your tenant before committing. It does NOT enforce (no Block/Enable). > [CmdletBinding()] param([switch]$Commit, [switch]$WhatIf, [string]$PolicyName = "EDLP-Windows-Pilot", [string]$PilotGroup = "SG-EDLP-Pilot", # pilot user/security group for the Devices scope [string]$LabelName = "Confidential", # sensitivity label the rules key on (adjust to your taxonomy) [string]$SimMode = "TestWithoutNotifications" # simulation/test mode; verify the exact value in your tenant)

How to run: PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\DLPWIN-Configure.ps1 -WhatIf # preview only (default is preview) .\DLPWIN-Configure.ps1 -Commit # create the policy + rules in SIMULATION .\DLPWIN-Configure.ps1 -Commit -PilotGroup "SG-EDLP-Pilot" -LabelName "Confidential" EXPECTED RESULTS With -Commit: policy 'EDLP-Windows-Pilot' exists in a Test/simulation mode, Devices location, pilot scope, with five Audit rules. Validate with DLPWIN-Validate.ps1 (expect Gate A PASS). Without -Commit: a printed plan and no changes. .NOTES White-label Microsoft Purview tooling. -WhatIf/preview by default; -Commit gates all writes. No credentials stored. Uses standard New-/Set-DlpCompliancePolicy and New-/Set-DlpComplianceRule; NO invented cmdlets. The exact endpoint-restriction Setting names/values and the simulation Mode value EVOLVE — the script uses documented names and PRINTS them for you to verify with `Get-Help New-DlpComplianceRule -Full` against your tenant before committing. It does NOT enforce (no Block/Enable). > [CmdletBinding()] param([switch]$Commit, [switch]$WhatIf, [string]$PolicyName = "EDLP-Windows-Pilot", [string]$PilotGroup = "SG-EDLP-Pilot", # pilot user/security group for the Devices scope [string]$LabelName = "Confidential", # sensitivity label the rules key on (adjust to your taxonomy) [string]$SimMode = "TestWithoutNotifications" # simulation/test mode; verify the exact value in your tenant)

DLPWIN-Discovery.ps1

Purpose: Read-only baseline discovery for Windows Endpoint DLP. Inventories the DLP compliance policies and rules already in the tenant, flags which cover the Devices (endpoint) location and their mode (simulation/test vs enable), and lists the sensitivity labels and sensitive information types the endpoint rules could key on. This is the "what already protects the endpoints" baseline before you build anything. Changes nothing. HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\DLPWIN-Discovery.ps1 .\DLPWIN-Discovery.ps1 -IncludeSensitiveTypes EXPECTED RESULTS A CSV baseline (.\DLPWIN-Discovery.csv) of DLP policies (with a DevicesLocation flag + Mode), DLP rules, and labels; with -IncludeSensitiveTypes, sensitive info types to .\DLPWIN-SensitiveTypes.csv. A tenant with NO Devices-location policy is itself the first finding. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Cmdlets are standard Security & Compliance PowerShell; none are invented. Endpoint DLP is licensing-dependent — the script WARNS where a feature is not readable rather than assuming. Device ONBOARDING status is confirmed in the Purview portal (Settings > Device onboarding); there is no supported S&C PowerShell cmdlet to read onboarding here. > [CmdletBinding()] param([switch]$IncludeSensitiveTypes, [string]$OutPath = ".\DLPWIN-Discovery.csv", [string]$SensitiveOutPath = ".\DLPWIN-SensitiveTypes.csv")

How to run: PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\DLPWIN-Discovery.ps1 .\DLPWIN-Discovery.ps1 -IncludeSensitiveTypes EXPECTED RESULTS A CSV baseline (.\DLPWIN-Discovery.csv) of DLP policies (with a DevicesLocation flag + Mode), DLP rules, and labels; with -IncludeSensitiveTypes, sensitive info types to .\DLPWIN-SensitiveTypes.csv. A tenant with NO Devices-location policy is itself the first finding. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Cmdlets are standard Security & Compliance PowerShell; none are invented. Endpoint DLP is licensing-dependent — the script WARNS where a feature is not readable rather than assuming. Device ONBOARDING status is confirmed in the Purview portal (Settings > Device onboarding); there is no supported S&C PowerShell cmdlet to read onboarding here. > [CmdletBinding()] param([switch]$IncludeSensitiveTypes, [string]$OutPath = ".\DLPWIN-Discovery.csv", [string]$SensitiveOutPath = ".\DLPWIN-SensitiveTypes.csv")

DLPWIN-Report.ps1

Purpose: Read-only operational + evidence report for Windows Endpoint DLP. Summarizes the DLP policies that cover the Devices location and their mode (simulation vs enforce), and — where the tenant/licensing exposes it — exports recent ENDPOINT activity from Activity Explorer (USB/print/clipboard/network/ cloud egress) so you can see what the policy is recording. Changes nothing. HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\DLPWIN-Report.ps1 .\DLPWIN-Report.ps1 -Days 7 EXPECTED RESULTS A console summary + .\DLPWIN-PolicyModes.csv (policies + mode + Devices flag) and, where available, .\DLPWIN-EndpointActivity.csv (Activity Explorer endpoint events). If activity export is not available in your tenant/licence, the script WARNS and reports policy modes only. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Uses Get-DlpCompliancePolicy and Export-ActivityExplorerData (Security & Compliance PowerShell); none invented. Activity Explorer export is licensing-dependent — the script degrades gracefully (warns) rather than assuming availability. > [CmdletBinding()] param([int]$Days = 7, [string]$ModesOutPath = ".\DLPWIN-PolicyModes.csv", [string]$ActivityOutPath = ".\DLPWIN-EndpointActivity.csv")

How to run: PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\DLPWIN-Report.ps1 .\DLPWIN-Report.ps1 -Days 7 EXPECTED RESULTS A console summary + .\DLPWIN-PolicyModes.csv (policies + mode + Devices flag) and, where available, .\DLPWIN-EndpointActivity.csv (Activity Explorer endpoint events). If activity export is not available in your

tenant/licence, the script WARNS and reports policy modes only. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Uses Get-DlpCompliancePolicy and Export-ActivityExplorerData (Security & Compliance PowerShell); none invented. Activity Explorer export is licensing-dependent — the script degrades gracefully (warns) rather than assuming availability. > [CmdletBinding()] param([int]$Days = 7, [string]$ModesOutPath = ".\DLPWIN-PolicyModes.csv", [string]$ActivityOutPath = ".\DLPWIN-EndpointActivity.csv")

DLPWIN-ScopeHealth.ps1

Purpose: Scope and health helper for Windows Endpoint DLP. Reports managed Windows-device signals from Microsoft Graph (a proxy for onboarding readiness), summarizes the pilot user/security group membership, and shows the current Devices-location scope of the Endpoint DLP policy — so you can see what the policy WOULD cover before you widen it. Read-only by default; any scope change is preview-only via -WhatIf (this tool does not itself widen scope — it prints the command for you to run under change control). HOW TO RUN PowerShell 5.1+ or 7+ with Microsoft Graph and (optionally) Security & Compliance PowerShell: Connect-MgGraph -Scopes "Device.Read.All","Group.Read.All" Connect-IPPSSession # optional, to read the policy scope .\DLPWIN-ScopeHealth.ps1 .\DLPWIN-ScopeHealth.ps1 -PilotGroup "SG-EDLP-Pilot" -WhatIf EXPECTED RESULTS A console summary: Windows managed-device count (Graph), pilot-group membership, and the policy's current Devices scope. With -WhatIf, a PREVIEW of the scope-widening command (nothing is changed). Confirm actual device ONBOARDING in Purview portal > Settings > Device onboarding (no PowerShell equivalent). .NOTES White-label Microsoft Purview tooling. Read-only / -WhatIf only. No credentials stored. Uses standard Microsoft Graph (Get-MgDeviceManagementManagedDevice / Get-MgGroup) and Get-DlpCompliancePolicy; none invented. Graph managed-device data is a PROXY for onboarding readiness — Endpoint DLP onboarding status itself is confirmed in the Purview portal. > [CmdletBinding()] param([switch]$WhatIf, [string]$PilotGroup = "SG-EDLP-Pilot", [string]$PolicyName = "EDLP-Windows-Pilot")

How to run: PowerShell 5.1+ or 7+ with Microsoft Graph and (optionally) Security & Compliance PowerShell: Connect-MgGraph -Scopes "Device.Read.All","Group.Read.All" Connect-IPPSSession # optional, to read the policy scope .\DLPWIN-ScopeHealth.ps1 .\DLPWIN-ScopeHealth.ps1 -PilotGroup "SG-EDLP-Pilot" -WhatIf EXPECTED RESULTS A console summary: Windows managed-device count (Graph), pilot-group membership, and the policy's current Devices scope. With -WhatIf, a PREVIEW of the scope-widening command (nothing is changed). Confirm actual device ONBOARDING in Purview portal > Settings > Device onboarding (no PowerShell equivalent). .NOTES White-label Microsoft Purview tooling. Read-only / -WhatIf only. No credentials stored. Uses standard Microsoft Graph (Get-MgDeviceManagementManagedDevice / Get-MgGroup) and Get-DlpCompliancePolicy; none invented. Graph managed-device data is a PROXY for onboarding readiness — Endpoint DLP onboarding status itself is confirmed in the Purview portal. > [CmdletBinding()] param([switch]$WhatIf, [string]$PilotGroup = "SG-EDLP-Pilot", [string]$PolicyName = "EDLP-Windows-Pilot")

DLPWIN-Validate.ps1

Purpose: Read-only validation of the Windows Endpoint DLP policy against the configuration gates in 05-testing.md (Gate A). Confirms the policy exists, covers the Devices location, is in a Test/simulation mode (during the pilot pass), is scoped to a pilot (not all devices), and has the five expected channel rules present and enabled. Prints PASS/FAIL per check. Changes nothing. HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\DLPWIN-Validate.ps1 .\DLPWIN-Validate.ps1 -PolicyName "EDLP-Windows-Pilot" EXPECTED

RESULTS A PASS/FAIL line per gate and an overall result. Expect all-PASS in the simulation pass. In the enforce pass, A3 (simulation) is expected to FLIP (policy Mode = Enable) BY DESIGN — that is noted, not an error. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Standard Get-DlpCompliance* cmdlets; none invented. Whether the policy SHOULD be in simulation depends on the phase — the script reports the mode and lets you judge against the phase you are in. > [CmdletBinding()] param([string]$PolicyName = "EDLP-Windows-Pilot", [string[]]$ExpectedRules = @("EDLP-Rule-USB","EDLP-Rule-Print","EDLP-Rule-Clipboard","EDLP-Rule-NetworkShare","EDLP-Rule-CloudEgress"))

How to run: PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\DLPWIN-Validate.ps1 .\DLPWIN-Validate.ps1 -PolicyName "EDLP-Windows-Pilot" EXPECTED RESULTS A PASS/FAIL line per gate and an overall result. Expect all-PASS in the simulation pass. In the enforce pass, A3 (simulation) is expected to FLIP (policy Mode = Enable) BY DESIGN — that is noted, not an error. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Standard Get-DlpCompliance* cmdlets; none invented. Whether the policy SHOULD be in simulation depends on the phase — the script reports the mode and lets you judge against the phase you are in. > [CmdletBinding()] param([string]$PolicyName = "EDLP-Windows-Pilot", [string[]]$ExpectedRules = @("EDLP-Rule-USB","EDLP-Rule-Print","EDLP-Rule-Clipboard","EDLP-Rule-NetworkShare","EDLP-Rule-CloudEgress"))